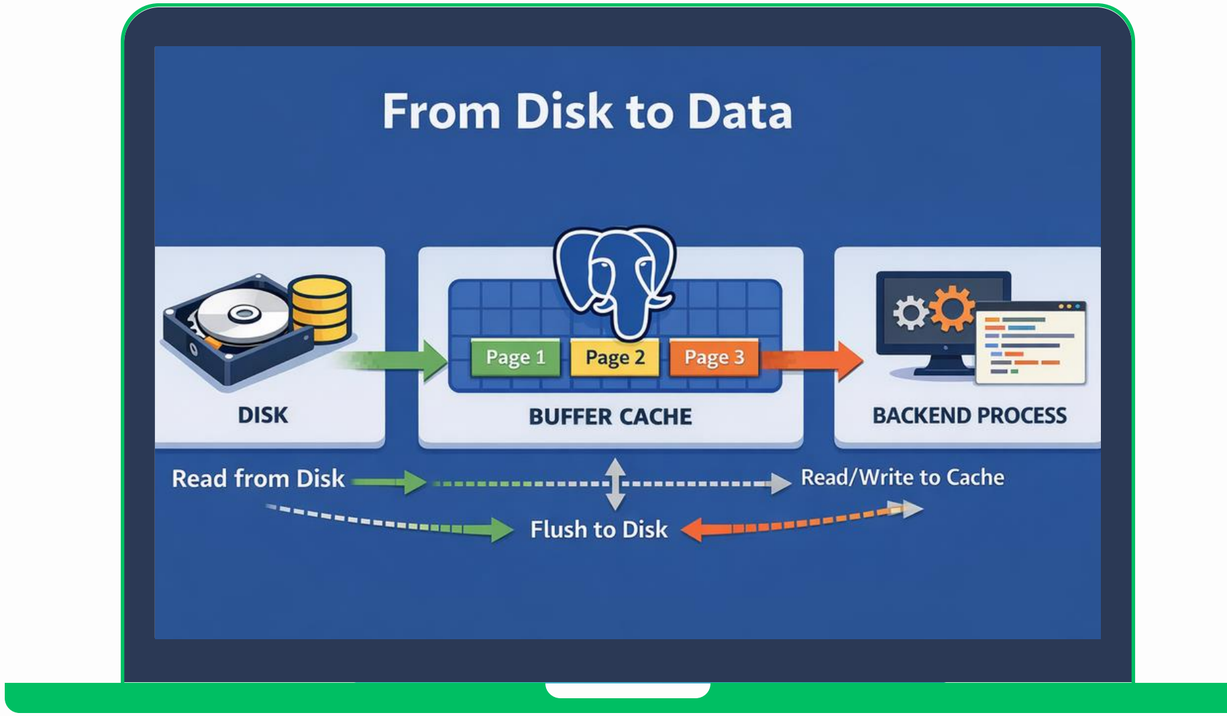
PGConf India

From Disk to Data: Exploring PostgreSQL's Buffer Management



By: Lalit Choudhry

purnabit

Introduction

Walkthrough of ,

- Buffer manager’s structure and job.
- Checking cached pages to reading from disk.
- How ring help with special workloads.
- How dirty pages are written out safely.

Why the Buffer Manager Matters

- Disk I/O is Incredibly Slow
- Databases are Always Working

Data gateway

it acts as a gateway, managing a special memory area called the Shared Buffer Pool.

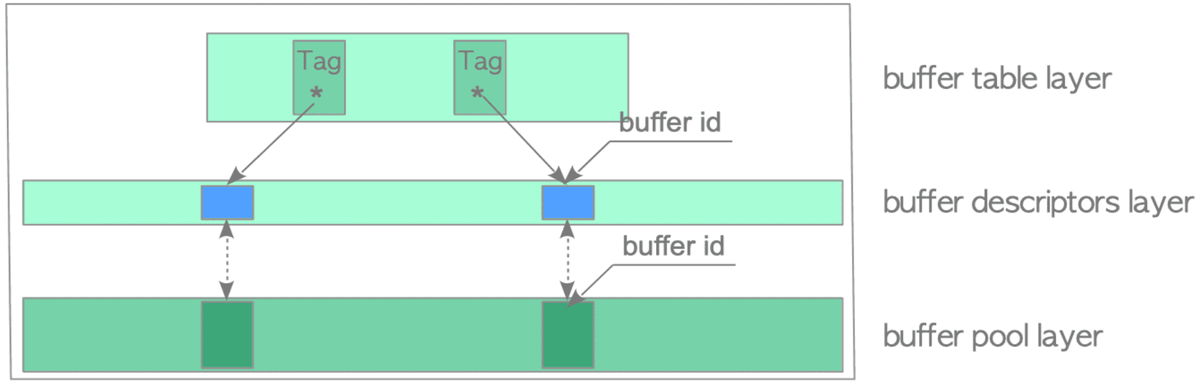
Fast Data Retrieval

Serve data directly from memory (cache) whenever possible.

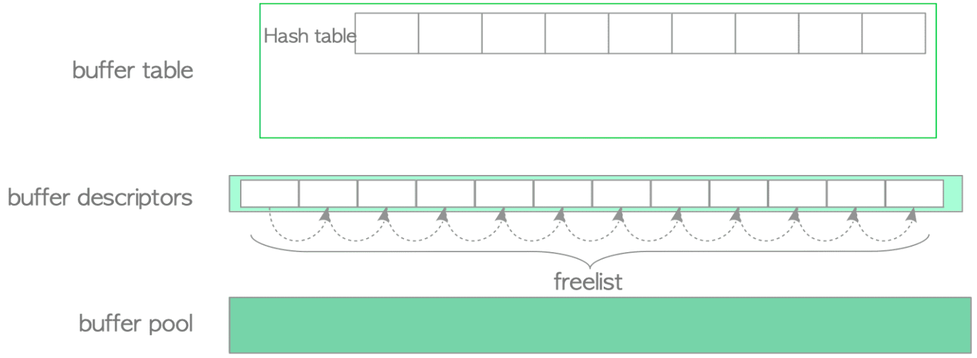
I/O Efficiency

Drastic reduction in slow disk I/O, making the database fast and efficient.

Buffer Manager Structure



https://www.interdb.jp/pg/pgsql08/02.html



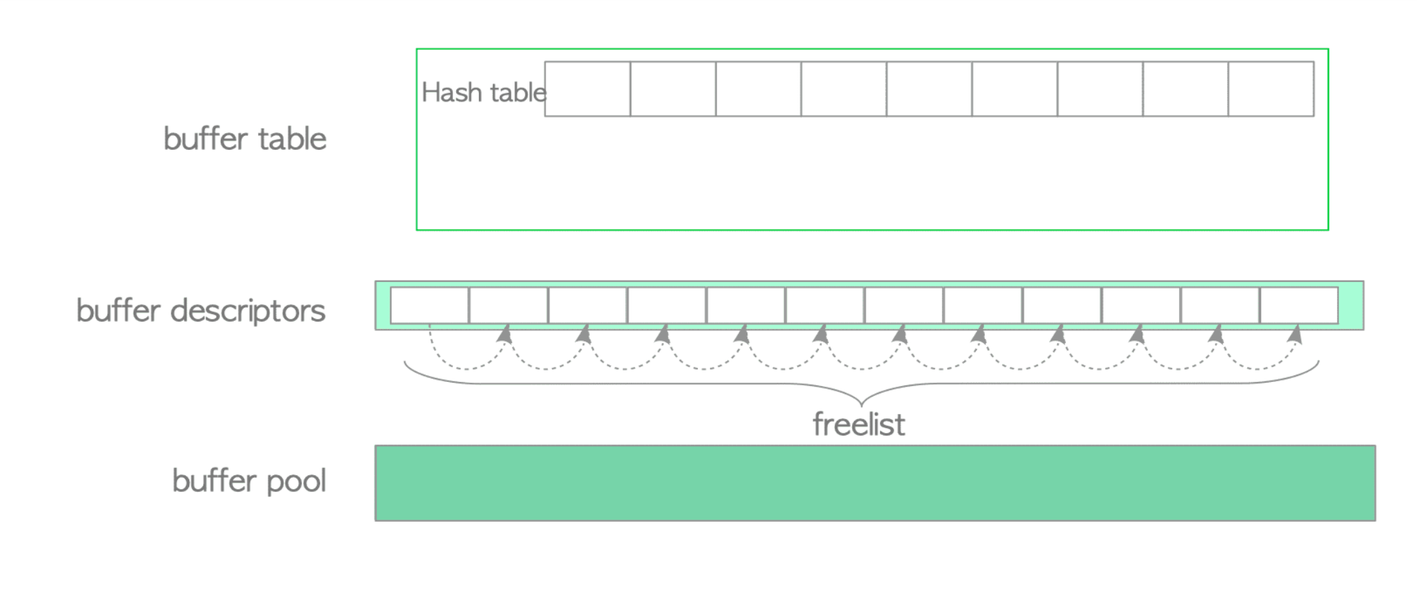
tblsOid dbOID relNumber forkNum blockNum

buffer_tag ‘{16821, 16384, 37721, 0, 7}’

BufferDesc
- buffer_tag,
- buf_id,
- content_lock
- freeNext pointer
- States, refcount and usage_count.
  - Empty
  - Pinned
  - Unpinned
  - Unpinned dirty
- nextVictimBuffer

Buffer pool
- buffer_id
- slot size is 8 KB (8kb page)

Buffer Manager Structure: [page loading]



Startup:
State of all buffer descriptors is 'empty'.

Load 1st page: Retrieve an empty descriptor from the top of the freelist, and pin it

https://www.interdb.jp/pg/pgsql08/02.html

Buffer Manager Structure

Backend process: Page Requested

Generate buffer_tag

Buffer Table

Lookup tag → Get buffer_id
Example: ‘Tag_A, id=1’

Buffer Descriptor

Access metadata at buffer_id

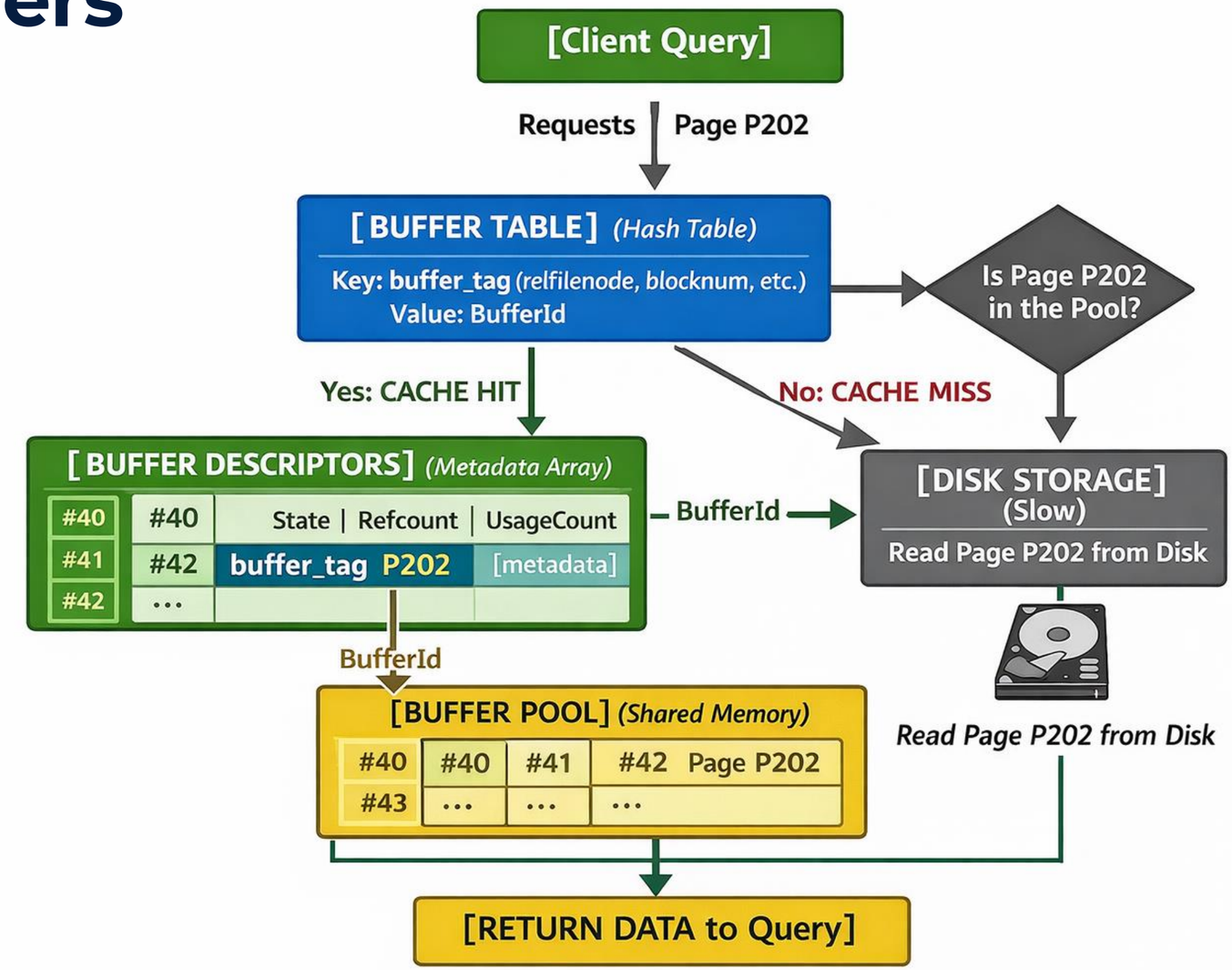
Buffer Pool

Retrieve actual page data at same buffer_id

Buffer Manager Structure

Component	Description	Purpose
Buffer Pool	Array storing actual data file pages. Each slot identified by a unique buffer_id	Physical storage of cached disk pages in memory
Buffer Descriptors	Parallel array with 1:1 correspondence to buffer pool slots. Each descriptor holds metadata for the page stored in its corresponding slot	Track page state (dirty, pin count, last access, etc.)
Buffer Table	Hash table mapping buffer_tags (unique page identifiers) to buffer_ids	Fast lookup to find if a page is already cached without scanning entire buffer pool

Visual flow: The Shared Buffer Pool & Its Managers



The Core Algorithm: How a Request is Served

1. **Request Arrives:** A backend process needs a specific page (e.g., Page_123).
2. **Hash Table Lookup:** It probes the hash table with the page ID.

- **Scenario: CACHE HIT**
  - The hash table returns the buffer slot number (e.g., Slot_5).
  - The process pins the buffer (increases its usage count to prevent it from being evicted).
  - It reads the data directly from memory and returns it.
  - **Result:** Fast, no disk I/O.

- **Scenario: CACHE MISS**
  - The page is not in the pool.
  - The process must find a free slot (or choose one to evict) using an algorithm (**clock sweep** - similar to LRU).
  - If the chosen victim slot contains a **dirty page** (modified data), it is first **flushed to disk**.
  - The process then reads Page_123 from disk into the chosen buffer slot.
  - The hash table is updated to point to the new location.
  - The page is pinned and the data is returned.

Handling Concurrency: Buffer Manager Locks

The Risk: Multiple processes try to modify the same buffer metadata simultaneously.

- The Solution: Lightweight Locks
  - BufMappingLock: Buffer manager's internal data structures (like the hash table).
  - Content Locks (Shared/Exclusive): Manage access to the contents of a buffer itself.

Flushing Dirty Pages Page [Replacement Algorithm: Clock Sweep]

1. nextVictimBuffer (pinned)
2. nextVictimBuffer (unpinned but its usage_count is 1)
3. nextVictimBuffer (unpinned and its usage_count is 0)

- **Background Writer:** A process that periodically scans the buffer pool and writes dirty (modified) pages to disk in the background. Its goal is to keep free/clean pages available and reduce the work during a checkpoint.
- **Checkpointer:** A process that ensures all dirty buffers up to a certain point are written to disk. This creates a recovery point.

Special Operations: Ring & Local Buffers

- **The Problem with Large Scans (e.g., Sequential Scan):**
  - Reading a huge table could flood the entire shared buffer pool with pages that are only used once, flushing out frequently used, valuable data.
- **The Solution: Ring Buffer**
  - For large sequential scans (or bulk writes like COPY), PostgreSQL allocates a small, temporary "ring" of buffers within the shared pool.
  - These pages are reused quickly and don't pollute the main cache. This protects the buffer pool from being thrashed.
- **Internal Operations: Local Buffer**
  - Some operations, like temporary tables used only within a single session, don't need to be shared.
  - PostgreSQL uses a **Local Buffer** in the backend process's own memory for these, avoiding the overhead of managing shared memory locks.

Summary:

- **Structure:** Shared Buffer Pool + Hash Table + Descriptors = The fast data cache.
- **Concurrency:** Lightweight Locks keep it safe for multiple users.
- **Algorithm:** Cache Hit (fast) vs. Cache Miss (find slot, maybe flush dirty page, load from disk).
- **Optimizations:** Ring Buffers protect the cache from large scans; Local Buffers speed up temp tables.
- **Durability:** Background Writer & Checkpointer ensure data is safely on disk without killing performance.

THANK YOU!

purnabit

Further Reading:

- The excellent resource this talk is based on: https://www.interdb.jp/pg/pgsql08.html
  Author: Hironobu SUZUKI
- PostgreSQL Official Documentation on Resource Consumption.

Get slides from :